



Software Engineering

UNIT 3- Lecture 10

Compiled by
Rajesh Banginwar

Department of Computer Science and Engineering

Acknowledgement: Prior year material, External university class material, authors of the prescribed textbooks and materials sourced online.



Software Engineering

Course Coordinator and Anchor (RR Campus): Prof. Rajesh Banginwar
Course Anchor (EC Campus) : Prof M Sheela Devi

Department of Computer Science and Engineering

Software Configuration Management

Software Configuration Management

- Software Configuration Management (SCM) is a software engineering discipline consisting of standard processes and techniques often used by organizations to manage the changes introduced into its software products. SCM helps in identifying individual elements and configurations, tracking changes, and version selection, control, and baselining.
- SCM is a Process to systematically organize, manage and control changes in documents, code and other entities that constitute a software product.
- SCM provides a mechanism for managing all changes in an efficient, cost-effective and timely manner.
- SCM also puts into place a framework for maintenance and support of a product.

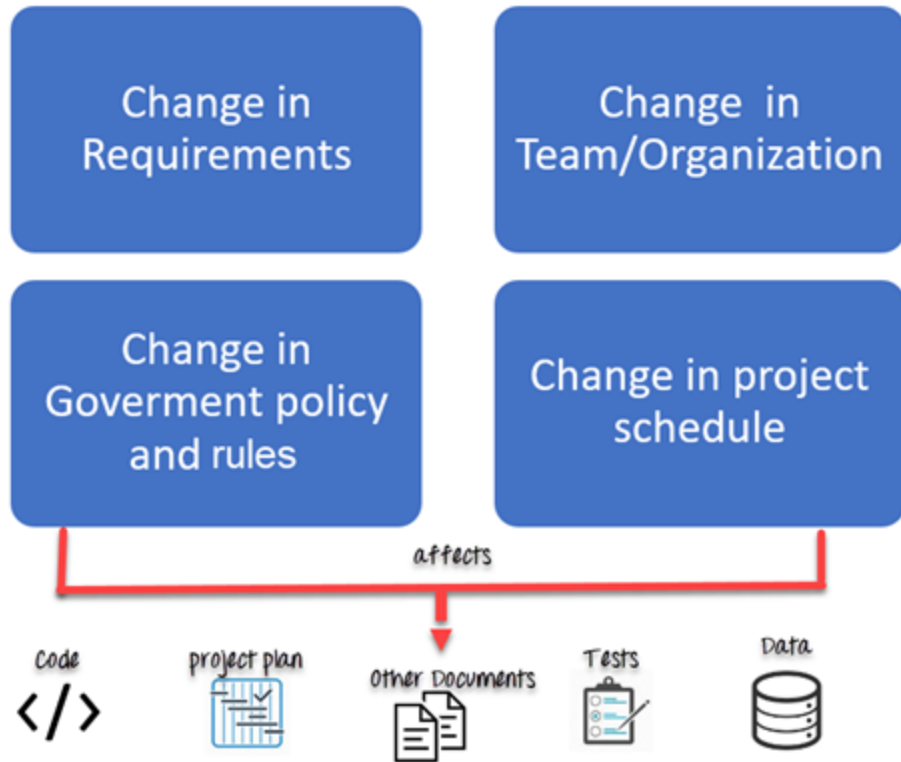
Goals of SCM

- Increase productivity by increased and planned coordination among the programmers and eliminate confusion and mistakes
- Identifying elements and configurations, tracking changes, version selection, control and baselining
- Avoid configuration related problems
- Effective management of simultaneous updating of source files
- Build management
- Defect tracking

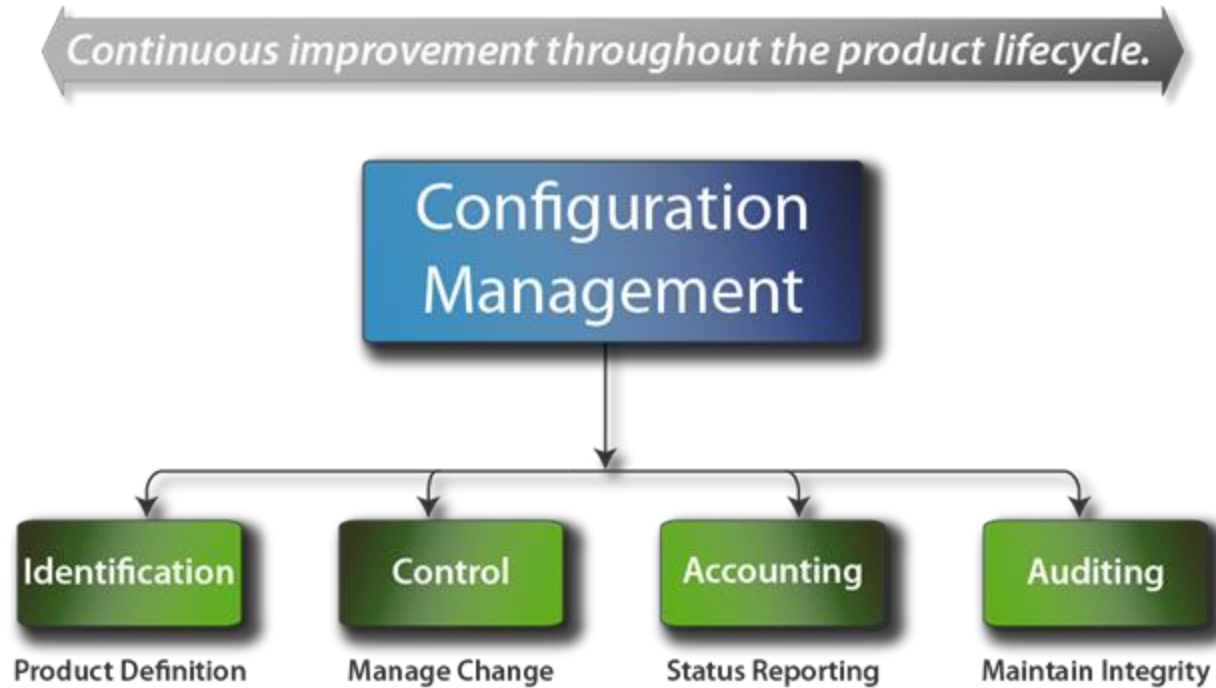
Why do we need SCM ?

- Software engineering in large products/projects with multiple people across locations need to be reliably managed for
 - Multiple people working concurrently on the same piece of software
 - Working on more than one version
 - Working on released systems
 - Changes in configuration due to changes in user requirements, budget, etc
 - Custom configured systems
 - Software must run on different machines
 - Coordination among stakeholders
 - Controlling costs in making changes

SCM Continued



Basic elements of SCM



Basic elements of SCM

SCM Element	Method
Software Configuration Item identification	Define baseline components
Software Configuration Control	Mechanism for initiating, preparing, evaluating, approving or disapproving all change proposals.
Software Configuration Auditing	Mechanism for determining the degree that the current state of a software system reflects its baselines (requirements and planning documents)
Software Configuration status accounting	Mechanism for maintaining a record of how a system has evolved, and the state of a system relative to published documents and written agreements.

SCM in Scrum-Agile Approach

- SCM is the responsibility of the “whole team” and is automated as much as possible.
- The definitive versions of components are held in a shared project repository.
- Developers copy versions from the repository into their workspace, make changes to the code and use system-building tools to create a new system on their computer.
- Once the changes are tested, the modified components are pushed to the project repository.
- Versions of the modified code are available to all team members.

Benefits of SCM

- Permits orderly development of software configuration items
- Ensures the orderly release and implementation of new or revised software products
- Ensures only approved changes to both new and existing software products are implemented and deployed
- Ensures that software changes are implemented in accordance with approved specifications
- Ensure that the documentation accurately reflects updates
- Evaluates and communicates the impact of changes
- Prevents unauthorized changes from being made

Configuration Management Roles

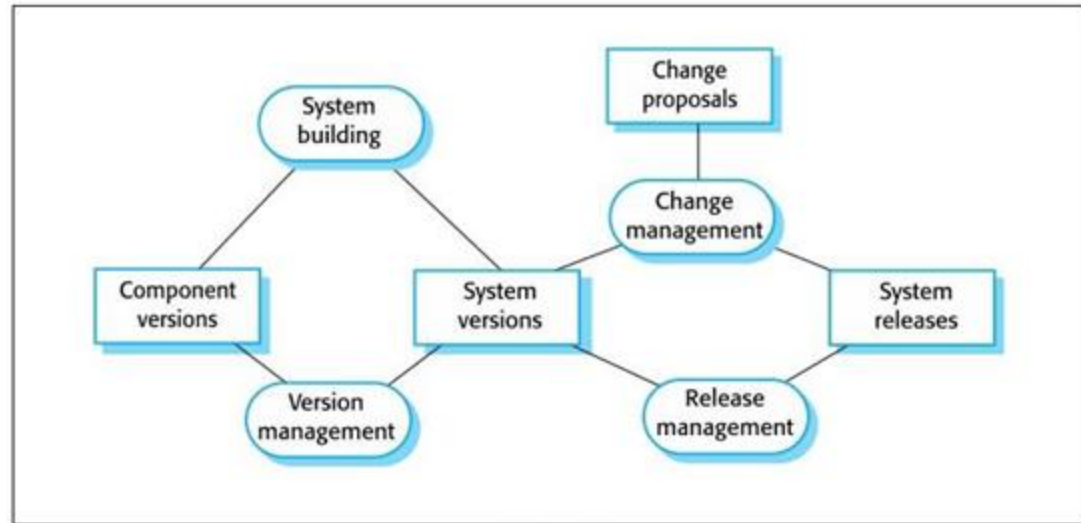
- Configuration Manager
 - Identify configuration items.
 - Define procedures for creating and promotions and releases.
- Developer
 - Create versions triggered by change requests or normal developmental activities.
 - Checks in changes and resolves conflicts.
- Auditor
 - Validate processes for selection and evaluation of promotions for release.
 - Ensures consistency and completeness.
- Change Control Board Member
 - Responsible for approving or rejecting change requests

Software configuration Management Planning

- Planning is done by a Configuration manager and starts during the early phases of the project
- Results in a Software Configuration Management Plan
 - May follow a public or internal standard
 - Defines the Configuration Items and a naming scheme
 - Define who takes responsibility for Configuration Management procedures and creation of baselines
 - Defines policies for change control and version management
 - Describes tools to assist in Configuration Management and any limitations
 - Defines the configuration management database

Software Configuration Management - Activities

- Configuration item Identification
- Configuration Management Directories
- Baselining
- Branch Management
- Version Management
- Build Management
- Install
- Promotion Management
- Change Management
- Release Management
- Defect Management



Configuration Items

- **Configuration item**: Independent or aggregation of hardware, software or both, that is designated for configuration management and treated as a single entity .

- Software configuration items could be
 - All types of code files and Drivers for tests
 - Requirement, analysis, design, test and other non temporary documents
 - User or developer manuals
 - System configurations

Challenges associated with configuration items

- What items need Configuration control?
 - Some items must be maintained for the lifetime of the software. Similar to object modelling

- When to place entities under configuration control
 - Start too early, too much bureaucracy
 - Start too late, introduces chaos

SCM Directories

Programmer's Directory (Dynamic Library)

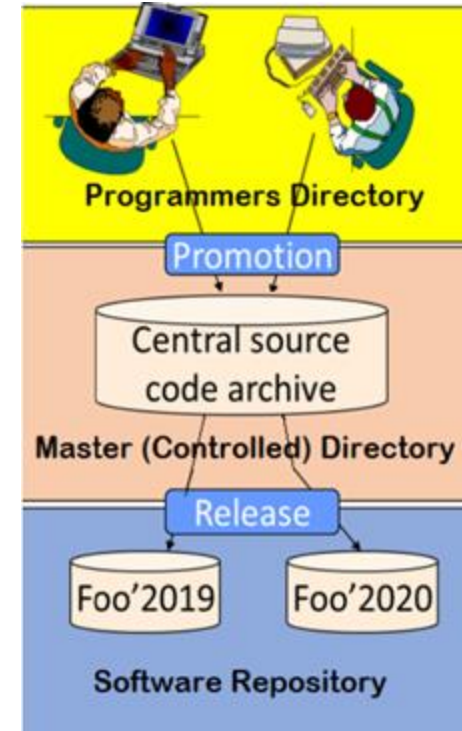
- Library for holding newly created or modified software entities
- Controlled by the programmer

Software Repository (Static Library)

- Archive for various baselines in general use
- Copies may be made available on request

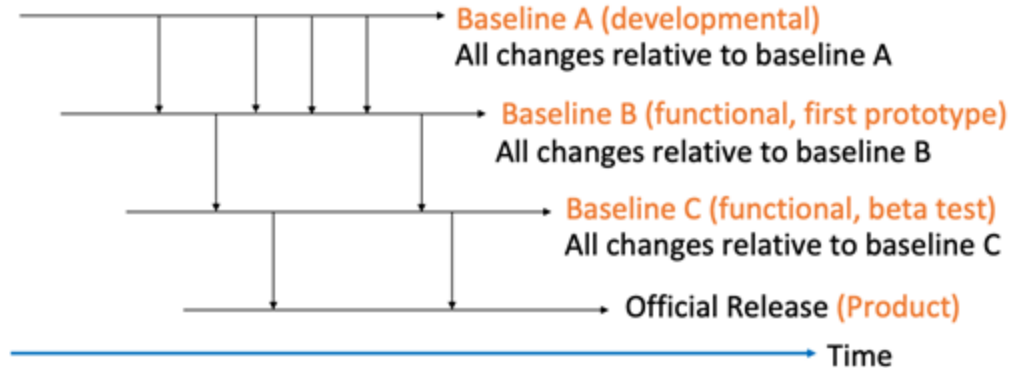
Master Directory (Controlled Library)

- Manages current baselines and for controlling changes
 - Entry is controlled, after verification
 - Changes must be authorized



Baselines

- **Baseline:** Specification or product that has been formally reviewed and agreed to by responsible management and serves as a basis and can only be changed on formal review
- As systems get developed, baselines get developed after a review
 - **Examples:**
 - **Baseline A:** APIs have been defined; Bodies of methods are empty
 - **Baseline B:** All data access methods are implemented and tested



SCM Activities

- **Codeline:** This term commonly describes **a group of files that evolve together over the life of the software application or application component**. A codeline might include source code, build scripts, images, compiled code, and other files that are required to create the application.

- **Branch:** copy or clone of all or a portion of the source code

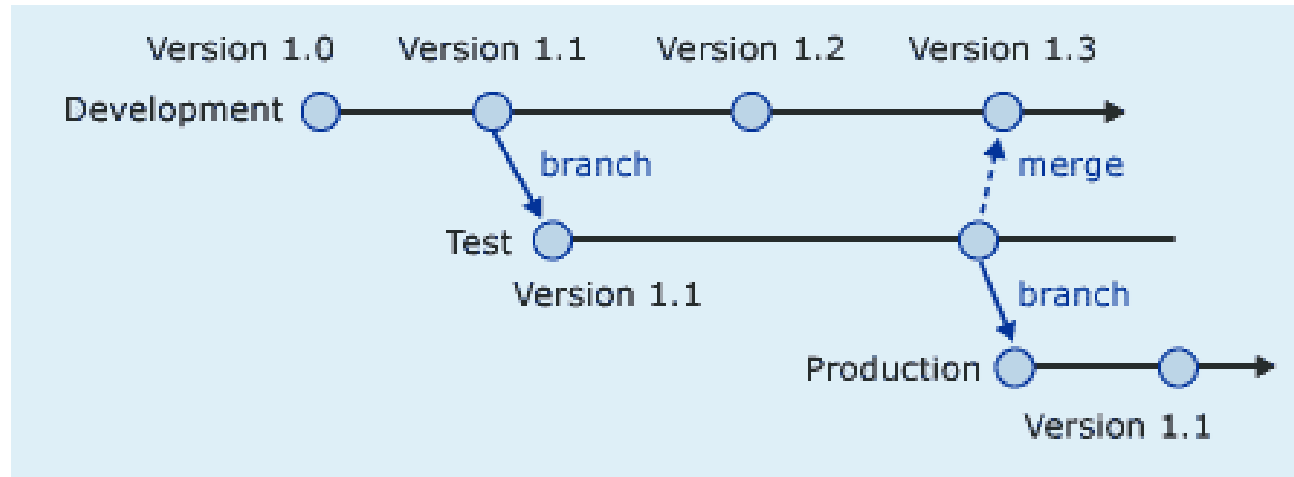
- Reasons for Branching
 - Supporting concurrent development
 - Capturing of solution configurations
 - Support multiple versions of a solution
 - Enable experimentation in isolation without impacting the stable version
 - Branching ensures that overall product is stable
 - Merging is bringing back and integrating changes over multiple branches
 - Frequent merging helps decrease the likelihood and complexity of a merge conflict.

SCM Activities – Branch Management

- Branching in version control and software configuration management, is the duplication of an object under version control (such as a source code file or a directory tree). Each object can thereafter be modified separately and in parallel so that the objects become different. In this context the objects are called **branches**. The users of the version control system can branch any branch.

- Many branching strategies which may be applied in combination
 - Single branch
 - Branch by customer or organization
 - Branch by developer or workspace
 - Branch by module or component
 - Branch Management entails having a well defined branching policy, owner for the code lines and usage of branches for release

Branching

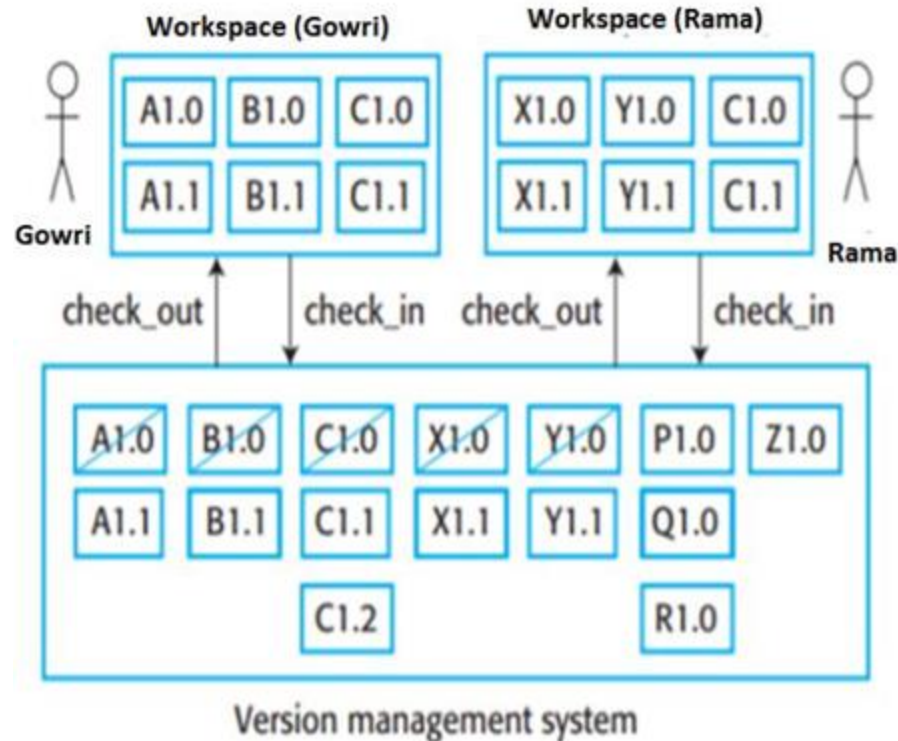


SCM Activities – Version Management

- **Version Management**: keeping track of different versions of software components and systems
- Usage of tools like git for version management
- Changes to a version are identified by a number, termed the revision number (7.5.5)
- 7 – Release number (defined by customer)
- 5 – Version number (defined by developer)
- 5 – Revision number (defined by developer)

- **Key Features of Version Control System**
 - Changes are attributable or traceable
 - Change history is recorded and can be reverted
 - Better conflict resolution
 - Easier code maintenance and quality monitoring
 - Less software regression
 - Better organization and communication

SCM Activities – Version Management



SCM Activities - Build Management

- **Build Management**: creating the application program for a software release by compiling and linking source code and libraries to build artifacts such as binaries or executables
- Done using tools like Make, Apache Ant, Maven, etc
- Compilation and linking of files in the correct order
- No need to recompile if no change in source code results in shorter build time

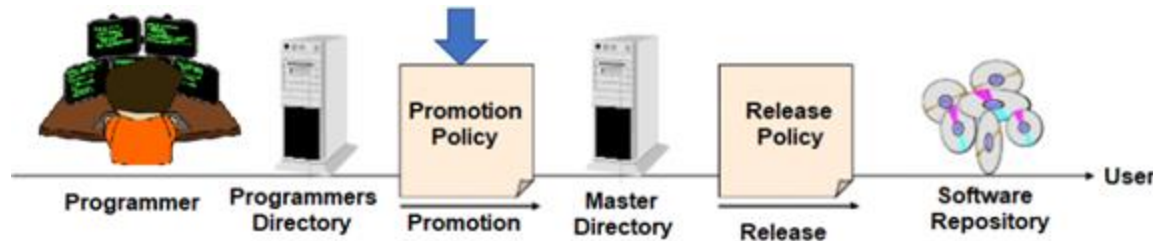
- **Build Process**
 - Fetching code from the source control repository
 - Compiling the code and checking the dependencies
 - Linking the libraries, code, etc
 - Running tests and building artifacts.
 - Archive logs and send notification emails
 - May result in version number change.

SCM Activities – Install Management

- Software installation is the first interaction with the customer .
- If install fails, **may result in negative perception**
- **Software installation**: placing multiple files containing executable code, downloading or copying from a repository, images, libraries, configuration files from the internet
 - Interaction with OS functions for validating the resources needed, permissions, versions, identifiers to ensure enforcement of licenses
 - May involve customizations for localizations
 - Sometimes may be automated using zip, shell scripts, InstallAware and Jenkins

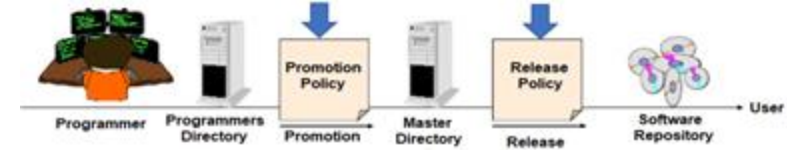
SCM Activities- Promotion Management

- Changes made by a programmer are only available in their environment and need to be promoted to a central master directory .
- Promotion is done based on certain promotion policies .
- Promotion could be based on baselining criteria which was planned for and would involve some amount of verification
- It would further be authorized and moved to the master directory
- Food for thought: Could compiled and executable code with a few warnings (not errors) be promoted to the master directory?



SCM Activities – Controlling changes

- Changes are controlled at two points
 - Promotion Policy
 - Release Policy



- Change Policies: the promotion and release policy is dictated via environment
 - **Informal**: good for research type environments and promotions
 - **Formal**: good for externally developed configuration item and for releases
- Change policies guarantee that each version, revision or release conforms to commonly accepted criteria and a consistent process is applied to how things are done
 - Enforced through engineering processes and tools
 - Audited to ensure conformation

SCM Activities – Release Management

- Movement of code to the customer and the software repository is done via release policies
- **Release Policy:** Gating quality criteria that is planned for and includes verification with metrics
- On meeting metrics, it would be authorized for a release
- **Release Management:** managing, planning, scheduling and controlling a software build through different stages and environments, testing and deploying software releases

- Version vs Release vs Revision
 - **Release:** formal distribution of an approved version
 - **Version:** Initial **release** or **re-release** of a configuration item associated with a complete compilation or recompilation of the item
 - Different versions have different functionality
 - **Revision:** change to a version that corrects only errors in the design/code, but does not affect the documented functionality.

SCM Activities – Bug or defect management

- **Bug:** Consequence of a coding fault
- **Defect:** variation or deviation from expected business requirements
- Driven through defect management tools like Bugzilla

- Defect Management Process

- Discover
- Reporting or logging into the tool with an unique identifier
- Validation
- Analysis and Categorization (Critical, High, Medium, and Low)
- Request and Approval for fix
- Resolution
- Verification by submitter
- Merging of the code
- Updating version number
- Plan for release of fix to customer
- Closure
- Reporting

SCM Tools – Types of S/W Config tools

- Source Code Administration
 - Used for source code control.
- Software Build
 - Used for building source code.
- Software Installation
 - Used for packaging and installing the products.
- Software Bug Tracking
 - Used for tracking bugs and changes.

SCM Tools – Source Code Administration Tools

➤ RCS

- Very old but in use.
- Used for version control system.

➤ CVS (Concurrent Version system)

- Based on RCS, allows concurrent working without locking.
- Has a web frontend.

➤ ClearCase

- Multiple servers, process modelling , policy check mechanism.

➤ GitHub

- Development platform for version control and project management.

Software Build

- **Software build** : Process of converting source code into standalone software artifact(s).
- Has a compilation process where build is converted to executable.

- Make
 - Automatically builds executable programs using Makefiles.
 - Makefiles derive target program.
- Cruise Control
 - Open source tool for continuous software builds.
- FinalBuilder
 - Automate build release management tool.
 - Easily define and maintain reliable build process.
- Maven
 - Software Project Management and comprehension tool.
 - Based on project object model.

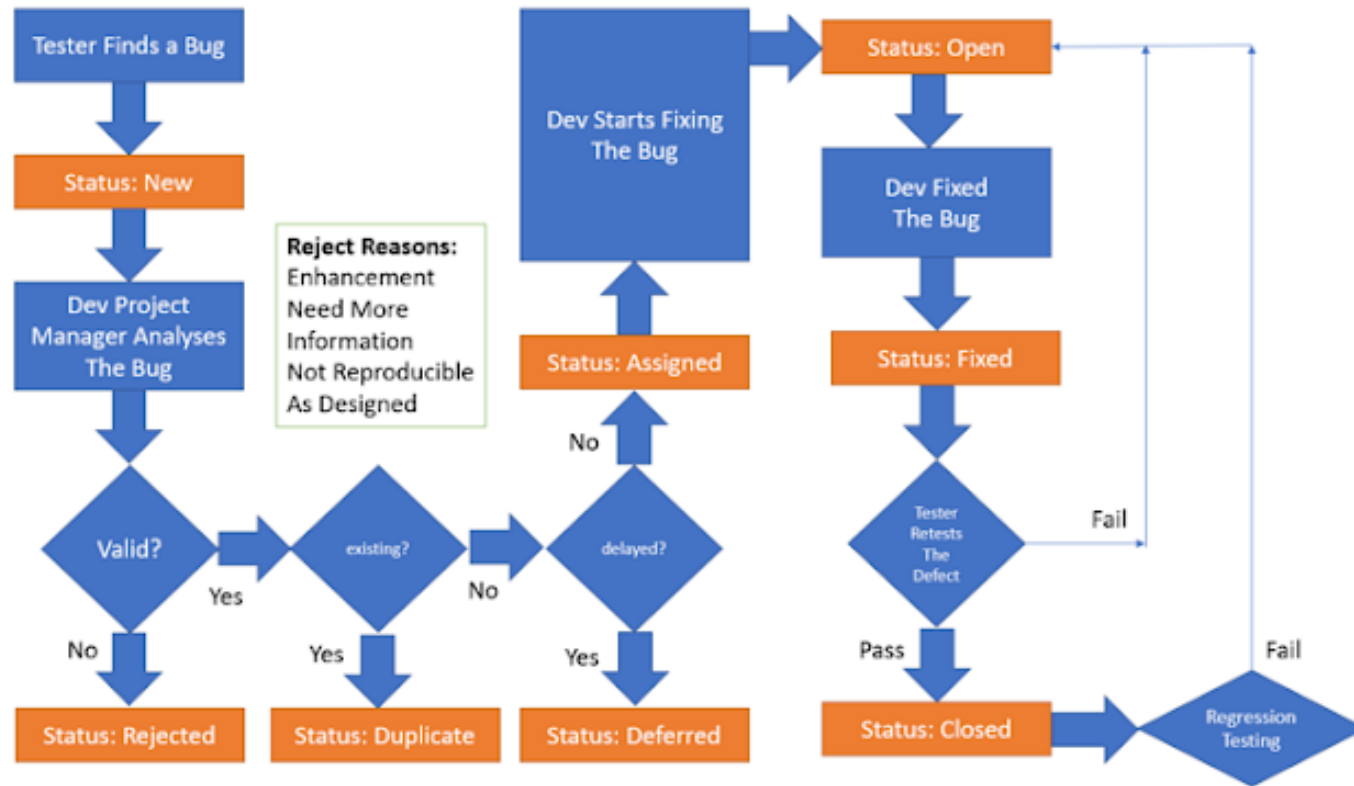
Software installation tools

- Cross platform tools that produce installers for multiple Operating systems
 - Installer: installs all the necessary files on the system
 - Could be customized to meet specific needs
- **Bootstrapper**: small installer that does the pre-requisites and updates the big bundle
- **DeployMaster** (Windows)
 - Distribute windows software or files via internet/CD/DVD
 - Works with different versions of windows.'
- **InstallShield**
 - Simplifies creation of windows installers, MSI packages and InstallScript installers for Windows.
 - De- facto for MSI Installation.
- **InstallAware** (Windows)
 - Windows installer platform for MS windows OS.
 - Supports Internet deployment.
- **WiseInstaller**
 - Configure and install Microsoft windows applications.

Software Bug or Defect Tracking Tools

- Software application that keeps track of reported software bugs in a software project
- Clear and centralized overview of development requests (bugs and improvements)
- Bug tracking systems support the lifecycle of a bug from logging to resolution
- Examples: Bugzilla, FogBugz, Trac Edgewall, Backlog, etc

Software Bug or Defect Lifecycle

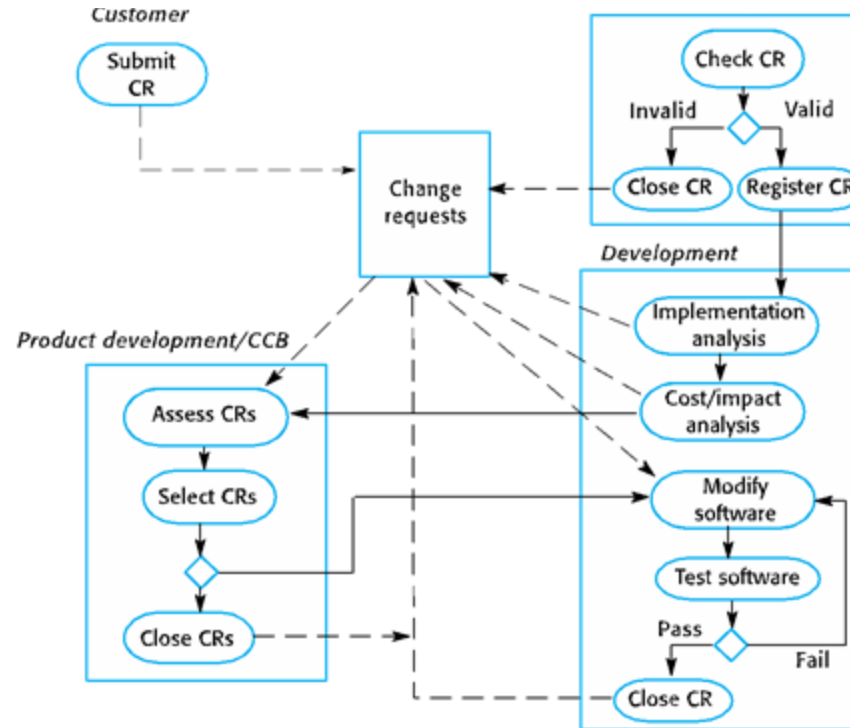


Change management

Change management

- Organizational needs and requirements change during the lifetime of a system, bugs have to be repaired and systems have to adapt to changes in their environment.
- Change management is intended to ensure that system evolution is a managed process and that priority is given to the most urgent and cost-effective changes.
- The change management process is concerned with analyzing the costs and benefits of proposed changes, approving those changes that are worthwhile and tracking which components in the system have been changed.

The change management process



A partially completed change request form (a)

Change Request Form

Project: SICSA/AppProcessing

Number: 23/02

Change requester: I. Sommerville

Date: 20/07/12

Requested change: The status of applicants (rejected, accepted, etc.) should be shown visually in the displayed list of applicants.

Change analyzer: R. Looek

Analysis date: 25/07/12

Components affected: ApplicantListDisplay, StatusUpdater

Associated components: StudentDatabase

A partially completed change request form (b)

Change Request Form

Change assessment: Relatively simple to implement by changing the display color according to status. A table must be added to relate status to colors. No changes to associated components are required.

Change priority: Medium

Change implementation:

Estimated effort: 2 hours

Date to SGA app. team: 28/07/12 **CCB decision date:** 30/07/12

Decision: Accept change. Change to be implemented in Release 1.2

Change implementor: **Date of change:**

Date submitted to QA: **QA decision:**

Date submitted to CM:

Comments:

Factors in change analysis

- The consequences of not making the change
- The benefits of the change
- The number of users affected by the change
- The costs of making the change
- The product release cycle

Change management and agile methods

- In some agile methods, customers are directly involved in change management.
- The propose a change to the requirements and work with the team to assess its impact and decide whether the change should take priority over the features planned for the next increment of the system.
- Changes to improve the software improvement are decided by the programmers working on the system.
- Refactoring, where the software is continually improved, is not seen as an overhead but as a necessary part of the development process.

Software Build of Materials (SBOM)

What is a Software Bill of Materials (SBOM)?

- A Software Bill of Materials (SBOM) is a detailed inventory of all software components, libraries, and modules—both open-source and proprietary—that are used in a software product. It's like a "parts list" in manufacturing but for software.
- **Purpose:** The SBOM provides transparency about what's inside a software package. It allows development teams, security analysts, and legal teams to identify dependencies, potential vulnerabilities, and licensing constraints.
- **Example Analogy:** In the same way a car manufacturer tracks every part that goes into a vehicle, an SBOM helps software manufacturers track every code dependency.
- **Typical Contents:** Package name, version, origin, checksum, license, relationship to other components.

➤ **Security Benefits:**

- When a vulnerability (e.g., Log4Shell) is discovered, SBOMs allow organizations to quickly identify if their software includes the vulnerable component.
- Enables proactive patch management and risk mitigation by identifying transitive dependencies.

➤ **Compliance & Licensing:**

- Helps track open-source components and their licenses (e.g., GPL, MIT, Apache) to ensure legal compliance.
- Critical for avoiding legal risks from unintentional use of incompatible licenses.

➤ **Traceability & Audit Readiness:**

- SBOMs make it easier to answer questions like: “Which version of libxyz are we using?” or “When did this component get introduced?”
- Regulatory bodies (e.g., in the US) are starting to mandate SBOMs for government and critical infrastructure software.

SBOM Standards, Tools

➤ Standards:

- **SPDX (Software Package Data Exchange):** An open standard by the Linux Foundation. Widely adopted and supported by many tools.
- **CycloneDX:** A lightweight SBOM format by the OWASP foundation, suitable for security-focused workflows.
- **SWID (Software Identification Tags):** ISO standard mainly used in enterprise asset management.

➤ Tooling Support:

- **Syft:** CLI tool to generate SBOMs in multiple formats.
 - Example: `syft myapp:latest -o cyclonedx-json`
- **Trivy:** Can scan containers for vulnerabilities and also output SBOMs.
- **OWASP Dependency-Check:** Performs vulnerability scanning and generates reports.
- **GitHub's Native SBOM Integration:** GitHub Actions can now auto-generate SBOMs during CI builds.

Example SBOM Entry (CycloneDX)

```
{  
  "bomFormat": "CycloneDX",  
  "components": [  
    {  
      "name": "log4j",  
      "version": "2.14.1",  
      "purl": "pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1",  
      "licenses": [ {"license": {"id": "Apache-2.0"}} ]  
    }  
  ]  
}
```



Software Engineering

Thank You

Department of Computer Science and Engineering

Attribution and Licensing

- Parts of this slideset include content adapted from the [CMU 17-313: Foundations of Software Engineering](#) course offered by Carnegie Mellon University.
- This material is used under the terms of the MIT License.
© 2025 Carnegie Mellon University.
- You are free to use, copy, modify, and distribute this material under the same license, provided that proper attribution is retained and this license notice is included in all copies or substantial portions of the material.
- Parts of the slideset include content derived from various Internet reports and articles.
- Parts of the slideset include content derived from other university syllabus and class material.